

TweetVerify: Detecting AI-Generated Political Tweets

Short-Text Modeling with Encoder Baselines and RLVR-Aligned LLM Judges

Yuchen Wang (1010441595), Xuanyi Lyu (1009343266), Zheyu Zheng (1008979580), and Benson Li (1007815376)

CSC490: Capstone Design Project - Final Report

December 2, 2025

Abstract

The proliferation of Large Language Models (LLMs) has democratized the production of high-fidelity synthetic text, creating an urgent challenge for the integrity of online political discourse. Malicious actors now leverage these tools to generate misinformation at scale, overwhelming traditional moderation systems on platforms like X (formerly Twitter). This report presents **TweetVerify**, a specialized detection system designed to distinguish between human-authored and AI-generated political tweets with high precision and low latency. Unlike generic detectors trained on long-form academic essays, TweetVerify is optimized for the noise, brevity, and slang inherent to social media. We comprehensively evaluate traditional supervised baselines—including RNN/LSTM and encoder-only Transformers (BERT, RoBERTa, DeBERTa-v3)—alongside a Reinforcement Learning with Verifiable Rewards (RLVR) framework, utilizing Group Relative Policy Optimization (GRPO) to align Qwen2.5-7B/14B-Instruct judge models. While our analysis demonstrates that RLVR improves the reasoning calibration of generative judges, our extensive evaluation reveals that fine-tuned encoder baselines (specifically RoBERTa) outperform LLM-based approaches on short-text classification. Consequently, TweetVerify deploys a RoBERTa-based primary classifier, with a hybrid Transformer ensemble used for offline analysis, on scalable AWS infrastructure provisioned with Terraform and SageMaker, incorporating rigorous rate-limiting and input validation to effectively balance sensitivity and specificity in the high-stakes domain of political verification.

1 Introduction and Problem Statement

1.1 Context and Motivation

As of late 2025, the digital landscape has been fundamentally altered by the ubiquity of generative AI. While tools like ChatGPT and Claude have enhanced productivity, they have simultaneously lowered the barrier to entry for operating automated influence campaigns. This is particularly critical in the political domain, where “bot farms” can now generate thousands of unique, coherent, and context-aware political tweets in seconds, simulating grassroots support (astroturfing) or spreading disinformation during election cycles.

The core threat is no longer just the existence of fake accounts, but the inability of users to distinguish machine-generated rhetoric from genuine human expression. When public discourse is flooded with synthetic content, trust in digital communities erodes. Users may dismiss genuine human advocacy as algorithmic noise, or conversely, accept hallucinated AI propaganda as fact.

1.2 Problem Statement

Current methods for verifying content authenticity are ill-equipped for the velocity and volume of modern social media. Manual verification initiatives, such as X’s Community Notes, rely on crowdsourced consensus. While accurate, these methods suffer from significant latency; a false claim can circulate to millions of users before a community note is appended days later.

Furthermore, automated detection remains a significant technical challenge due to the “**short-text problem**.” Most state-of-the-art detectors rely on statistical irregularities (perplexity and burstiness) that are easily measurable in long documents like essays. However, tweets, which are constrained by character limits and characterized by heavy use of emojis, hashtags, and informal grammar, often lack sufficient token density for these statistical signals to manifest reliably.

1.3 Scope and Objectives

TweetVerify aims to bridge this gap by developing an automated, real-time verification pipeline specifically for political discourse on Twitter. Our objectives are:

1. **Robust Detection:** Develop a model that outperforms general-purpose classifiers on short-text data by leveraging fine-tuned Transformer architectures and Reinforcement Learning alignment.
2. **Adversarial Resilience:** Ensure the system remains accurate against common evasion techniques, such as adversarial paraphrasing and “humanizing” AI text.
3. **Scalable Infrastructure:** Deploy the solution as a production-grade API designed to handle high-throughput traffic with low latency, leveraging AWS-based orchestration and infrastructure.

2 Related Work

The field of AI text detection has expanded rapidly, yet a gap remains between academic benchmarks and industrial applicability for social media.

2.1 Academic Approaches

Early research in text classification relied heavily on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs) architectures to capture sequential dependencies. As demonstrated in our preliminary evaluations, while LSTMs offer decent generalization, they struggle with the complex contextual nuances of modern generative models.

Transformer-based architectures now dominate the academic landscape. **BERT** (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019) revolutionized this domain by enabling deep bidirectional context modeling. Our preliminary experiments confirmed that fine-tuning BERT on a mixture of curated political tweets, the *Sentiment140* sentiment corpus, and the DAIGT authorship dataset substantially outperforms RNN-based baselines.

However, standard supervised fine-tuning (SFT) often causes models to overfit specific generator artifacts. This limitation motivated a shift toward detection methods that exploit *distributional* rather than *stylistic* signals. A prominent early line of work focuses on zero-shot statistical detection: **DetectGPT** (Mitchell et al., 2023) demonstrates that curvature-based deviations in log-probability space provide a reliable indicator of machine-generated content, while **DetectLLM** (Su et al., 2023) shows that token-level rank instability is a strong cross-model signal. Both works highlight that the most informative cues for AI authorship are *statistical artifacts* rather than semantic coherence.

This perspective aligns with more recent findings by Miresghallah et al. (2024), who demonstrate that *smaller* language models can outperform large autoregressive models as universal detectors in zero-shot settings. Their study shows that encoder-style models and compact transformers more effectively capture generator-agnostic statistical patterns, whereas large LLMs over-rely on high-level semantics and consequently fail to detect modern, stylistically humanlike text. These insights directly motivate our decision to benchmark both encoder models and large-scale LLM judges.

Beyond classification-focused methods, recent research explores **Reinforcement Learning with Verifiable Rewards (RLVR)** for improving reasoning reliability. **Group Relative Policy Optimization (GRPO)** (Shao et al., 2024) demonstrated that reasoning performance in small models can be significantly enhanced by replacing PPO’s critic with relative-group baselines. Furthermore, Wen et al. (2025) showed that even binary rewards suffice to teach models to reason consistently. TweetVerify builds on this lineage by applying GRPO to authorship verification, aiming to produce a judge model that is both accurate and well-calibrated.

2.2 Industry Landscape and Gap Analysis

In the commercial sector, several tools have emerged, though most are misaligned with the distinct requirements of political short-text verification.

- **Turnitin and GPTZero:** These platforms remain industry leaders for academic integrity. They employ features such as perplexity and burstiness to detect AI plagiarism. However, their performance degrades significantly on short text (under 50 tokens) and in informal domains such as Twitter, which frequently contain slang, emojis, and user mentions.
- **Originality.ai:** This tool is optimized for SEO and web publishing rather than detecting political manipulation or coordinating influence campaigns.
- **Community Notes (X/Twitter):** While offering high-context, high-precision fact-checking, this is not an automated detection system but a human crowdsourcing protocol. As such, it suffers from scalability limitations and delayed reaction time.

TweetVerify differentiates itself by targeting the *short-text political domain*. Unlike commercial generalists, our pipeline incorporates Twitter-specific preprocessing (hashtags, mentions, conversation threading) and includes an experimental Qwen2.5-14B RLVR-trained judge model for robustness studies against adversarial political misinformation (Zhao et al., 2025).

3 Technical Approach, Solution, and Design

3.1 Dataset

We utilize the *Global Political Tweets* dataset (Suresh, 2021) as the primary source of real-world political discourse. The dataset comprises a vast collection of tweets covering global political topics, curated from diverse regions and timeframes. It provides a heterogeneous mixture of political stances, linguistic styles, and topical domains, making it ideal for training robust authorship classifiers. The dataset’s scale allows our model to capture cross-lingual and cross-domain variations in political messaging. Furthermore, we leveraged the Twitter API to scrape recent political tweets, augmenting the dataset with real-time information to ensure exposure to contemporary linguistic patterns and evolving narratives.

3.2 Data Preprocessing Pipeline

Our data preprocessing pipeline consists of three major stages: (1) raw political tweet extraction and cleaning, (2) high-quality human tweet filtering, and (3) synthetic variant creation and curation.

3.2.1 Stage 1: Raw Tweet Extraction and Cleaning

We begin by extracting political tweets from the raw dataset. The cleaning script `cleaning_political_tweets.py` removes empty rows, selects relevant columns (user description and tweet text), and filters invalid entries. All valid rows are persisted to `cleaned_output.csv`.

3.2.2 Stage 2: Human Tweet Quality Filtering

To establish a clean human baseline, we implemented an asynchronous filtering pipeline, `filter_good_tweets.py`, which adaptively groups tweets into batches of approximately 1100 tokens. The pipeline utilizes a structured prompt to instruct `GPT-4o-mini` to classify each tweet. The system flags tweets as *BAD* if they contain advertising, self-promotion, spam, irrelevant content, or lack clear semantic meaning. The JSON output is parsed, misaligned labels are repaired, and all *GOOD* tweets are streamed into `good_tweets_async.csv`.

Subsequently, for the final extraction, the script `reformat_human_response_high_quality.py` normalizes the high-quality tweets into a curated file, `high_quality_human.csv`, adhering to the schema (text, label = 0).

3.2.3 Stage 3: AI Tweet Generation and Curation

(a) Political topic-constrained synonym substitution: To augment the dataset, a subset of high-quality human tweets was expanded into five stylistically consistent variants using a high-parallelism asynchronous pipeline (`creating_ai_tweets.py`), rate-limited to 480 RPM. We employed a strict prompting strategy that constrained `GPT-5-mini` to:

- Maintain the same political stance and viewpoint;
- Retain the original emotional intensity and tone;
- Preserve stylistic elements, including punctuation, capitalization, and emoji usage.

Following generation, the pipeline parses the JSON output, validates its structural integrity to ensure exactly five variants are generated per seed tweet, and records the verified entries in `ai_generated_realtime.csv`.

(b) Adversarial paraphrasing: A selected subset of high-quality human tweets was expanded into five adversarial variants using the script `adversarial_ai_tweets.py`. We utilized a specialized prompting strategy that constrained `GPT-5-mini` to:

- Preserve the original political stance and semantic meaning;
- Rewrite aggressively to mimic authentic, informal Twitter syntax;
- Intentionally inject slang, typos, abbreviations, inconsistent capitalization, and irregular punctuation;
- Explicitly optimize for evading automated text classifiers by simulating human imperfection.

The resulting variants underwent the same validation pipeline before being appended to the `ai_generated_realtime.csv` dataset.

Finally, the script `reformat_and_filter_ai_response.py` normalizes the raw AI-generated variants into a definitive curated dataset, `ai_generated.csv`, adhering to the schema: (text, label = 1).

3.3 Models

We organize our models into two broad categories. First, we train a family of *supervised baseline detectors* that operate purely on labeled examples of human- and AI-generated tweets. These include classical recurrent networks (RNNs and LSTMs), pretrained encoder-only Transformers (BERT, RoBERTa, DeBERTa), a feature-augmented RoBERTa variant, a zero-shot Fast-DetectGPT detector, and simple ensembles. Second, we develop our *comparison model*, an RLVR-aligned Qwen2.5-14B-Instruct and a Qwen2.5-7B-Instruct judge trained with Group Relative Policy Optimization (GRPO) on verifiable correctness signals. The baselines provide interpretable performance anchors and ablation points against which we can quantify the value of RL-based alignment.

3.3.1 Classical Supervised Baselines: RNN and LSTM

As a first point of comparison, we train Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks as lightweight sequence-classification baselines. These models are efficient to train, require modest compute, and remain widely used in applied NLP systems. RNN-based architectures are naturally suited for sequential data because they process tokens in order and retain contextual information through their hidden states. The architecture of our recurrent baselines is summarized below:

1. **Embedding Layer:** We fine-tune a Word2Vec model on our dataset to obtain embeddings that reflect the emoji-rich, informal vocabulary of political tweets. These pre-trained embeddings, plus a zero vector for padding, initialize the embedding layer and remain fixed during training.
2. **Recurrent Layers (RNN):** A bidirectional RNN with hidden size 300 is used to capture local sequential dependencies in the text.
3. **Recurrent Layers (LSTM):** For the LSTM baseline, we replace the vanilla recurrent cell with a two-layer bidirectional LSTM with hidden size 256, allowing the model to preserve information over longer contexts through input, forget, and output gates.
4. **Dropout Layer:** A dropout rate of 0.2 is applied after the recurrent layers to reduce overfitting and improve generalization.
5. **Output Layer:** A fully connected layer maps the aggregated bidirectional hidden states to logits over the two classes (human vs. AI).

Although RNNs are simple and intuitive, they often struggle with long-term dependencies due to vanishing gradients. LSTMs partially address this limitation with gating mechanisms that regulate information flow, enabling the model to retain relevant context over longer sequences. In our experiments, both models serve as *classical baselines* that help quantify the gains from moving to Transformer-based and RLVR-based approaches.

3.3.2 Transformer Supervised Baselines: BERT, RoBERTa, and DeBERTa

To build stronger baselines, we adopt three representative encoder-only Transformers: BERT, RoBERTa, and DeBERTa. These models have demonstrated state-of-the-art performance on a wide range of text classification tasks and are particularly well suited to capturing subtle contextual differences between human-written and AI-generated tweets.

We treat all three Transformer variants as *supervised baselines* that are trained using standard cross-entropy on our curated dataset. Concretely:

1. **Tokenization and Embedding:** Tweets are tokenized using the respective pretrained tokenizer for each model. Input sequences are padded or truncated to a fixed length, and special tokens (such as [CLS] and [SEP] for BERT) are inserted according to the model’s convention.
2. **Transformer Encoder Layers:** Each model is built from stacked Transformer encoder blocks with multi-head self-attention and position-wise feed-forward sublayers, enabling global context modeling over the entire tweet.

3. **Dropout Layer:** Given the higher capacity of these models, we apply a dropout rate of 0.3 after the encoder to mitigate overfitting.
4. **Classification Head:** For BERT, the hidden state associated with the [CLS] token is fed into a linear classification head. RoBERTa and DeBERTa use their respective pooled representations, which are similarly mapped to logits over the two classes.

In this work, fine-tuned BERT, RoBERTa, and DeBERTa serve as our *strong supervised baselines*. They allow us to compare the RLVR-aligned judge against well-established Transformer classifiers trained under the standard supervised-learning paradigm.

3.3.3 Feature-Augmented RoBERTa Model

Inspired by the work of Kumar et al. (2023), we incorporated additional handcrafted stylistic and statistical features into the output representation of our Transformer-based model to examine whether they improve detection performance. The specific features extracted from each tweet are described below:

- **PPL (AI-generated perplexity score):** A likelihood-based estimate indicating how machine-like the text appears. Lower perplexity values generally correlate with AI-generated content.
- **Emoji count:** The number of emoji symbols appearing in the tweet. Emoji usage often reflects expressive intent and informal stylistic cues that differ between human and AI-generated content.
- **Caps ratio:** The proportion of uppercase characters in the tweet, capturing emphasis and stylistic patterns that differ between human-written and AI-generated text.
- **Punctuation count:** The total number of punctuation marks, reflecting writing rhythm, structural complexity, and formatting style.
- **Dash count:** The number of dash characters (e.g., “—” or em-dash “—”) appearing in the tweet. Large language models often employ dashes more frequently than human writers, making dash usage a useful stylistic cue for distinguishing AI-generated text.

The perplexity scores are computed using an external LLM (Qwen3-14B), while the remaining features are extracted manually using Python. These five handcrafted features are first passed through a fully connected layer and then concatenated with the [CLS] embedding produced by the Transformer-based encoder. The resulting joint representation is subsequently fed into an additional MLP classifier to produce the final confidence score.

3.3.4 Zero-Shot Method

Zero-shot methods are known for their training-free nature, making them robust and largely independent of dataset quality. They also mitigate concerns related to unseen examples or distribution shift. We adopt the approach proposed by Bao et al. (2024), which computes a conditional probability curvature that measures how strongly the model prefers the ground-truth tokens over nearby alternatives.

It is well known that during text generation, large language models tend to select, at each position j , the token with the highest probability in the next-token distribution. This causes the log-probability of the chosen (real) token to form a pronounced peak relative to other candidate tokens. In contrast, human writers choose words based on meaning, intent, and stylistic preference rather than likelihood, so the true token’s log-probability is typically much less sharply peaked compared to that of a model-generated token.

To quantify this distinction, we measure how strongly the model favors the real token over its nearby semantic neighbors using the Conditional Probability Curvature (CPC) defined by

$$d(x, p_\theta, q_\varphi) = \frac{\log p_\theta(x | x) - \tilde{\mu}}{\tilde{\sigma}}, \quad (1)$$

where

$$\tilde{\mu} = \mathbb{E}_{\tilde{x} \sim q_\varphi(\tilde{x} | x)} [\log p_\theta(\tilde{x} | x)], \quad (2)$$

$$\tilde{\sigma}^2 = \mathbb{E}_{\tilde{x} \sim q_\varphi(\tilde{x} | x)} \left[(\log p_\theta(\tilde{x} | x) - \tilde{\mu})^2 \right]. \quad (3)$$

$\tilde{\mu}$ denotes the expected score of samples $\tilde{x}$ generated by the sampling model $q_\varphi(\cdot | x)$, and $\tilde{\sigma}^2$ is the expected variance of the scores. A larger score is associated with higher confidence of the text being generated by AI. In our

case, we use GPT2-XL as the sampling model to generate alternative candidate tokens. For each token, we sample 1,000 perturbations to estimate the curvature-based score. The decision threshold for classification is fine-tuned on the validation set, after which the same threshold is applied to the test set to evaluate the detector’s performance.

3.3.5 Ensemble Method

To further improve detection performance, we also employ an ensemble method that aggregates the predictions of multiple independently trained models. Ensemble strategies are widely used in practice, such as in Kaggle competitions, due to their effectiveness in combining the strengths of individual models and reducing variance in the final prediction. We mainly experiment with two ensemble strategies: weighted averaging and majority voting.

- **Weighted averaging:** Each model produces a probability score for the input, and the final prediction is obtained by taking a weighted average of these probabilities. The weights are tuned on the validation set to balance the relative strengths of different models.
- **Majority voting:** Each model outputs a binary prediction (AI-generated or human-written), and the final label is determined by majority vote among all models. This method helps reduce the influence of any single model’s bias and improve robustness.

3.3.6 RLVR-Aligned Qwen2.5-7B/14B Judge: Theoretical Formulation

While the supervised and zero-shot baselines treat tweet classification as a standard discriminative learning problem, our alternative model casts authorship verification as a reinforcement learning with verifiable rewards (RLVR) task. Intuitively, instead of minimizing cross-entropy loss on fixed labels, we directly optimize large language models (Qwen2.5-7B and 14B-Instruct) to maximize a binary reward that encodes whether its judgment about a tweet is *correct*.

Problem setup. Let $x \in \mathcal{X}$ denote a political tweet and $y^* \in \{0, 1\}$ its ground-truth label, where 0 represents human-written and 1 represents AI-generated. The model produces a textual judgment $a \in \mathcal{A}$, for example:

$$a = \text{“This tweet is AI-generated.”} \quad \text{or} \quad a = \text{“This tweet is human-written.”}$$

We treat the model as a conditional policy $\pi_\theta(a \mid x)$ parameterized by θ (the weights of Qwen2.5-14B). A deterministic mapping $f : \mathcal{A} \rightarrow \{0, 1\}$ extracts the implied binary decision $\hat{y} = f(a)$ from the natural-language judgment.

Verifiable reward. RLVR assumes that, given (x, y^*, a) , we can compute a *verifiable* reward $r \in \{0, 1\}$ without any further human annotation:

$$r(x, a, y^*) = \begin{cases} 1, & \text{if } f(a) = y^*, \\ 0, & \text{otherwise.} \end{cases}$$

This reward corresponds exactly to classification correctness: the model obtains reward 1 if and only if it correctly identifies the tweet as human or AI. Crucially, this removes the need for a separate learned reward model; the supervision signal is defined by the task itself.

Group Relative Policy Optimization (GRPO). Directly applying standard policy gradient methods (e.g. REINFORCE) leads to high-variance updates and unstable training for large language models. Following Shao et al. (2024) and Wen et al. (2025), we instead use Group Relative Policy Optimization (GRPO), which reduces variance by normalizing rewards within *groups* of sampled trajectories.

For a given tweet x , we sample a group of K candidate judgments $\{a_1, \dots, a_K\}$ from the current policy:

$$a_k \sim \pi_\theta(\cdot \mid x), \quad k = 1, \dots, K.$$

Each judgment a_k receives a binary reward $r_k \in \{0, 1\}$. GRPO defines a group-relative advantage:

$$A_k = r_k - \bar{r}, \quad \bar{r} = \frac{1}{K} \sum_{j=1}^K r_j,$$

which measures how much better or worse a particular judgment is compared to the other samples for the same

tweet. The GRPO objective for a single group is

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{K} \sum_{k=1}^K A_k \log \pi_{\theta}(a_k | x) + \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\theta_{\text{ref}}}(\cdot | x)),$$

where $\pi_{\theta_{\text{ref}}}$ is a frozen reference policy (the instruction-tuned Qwen2.5-14B before RLVR), and $\beta > 0$ controls the strength of a KL-regularization term that prevents the policy from drifting too far from its original capabilities.

The gradient update is then

$$\nabla_{\theta} \mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{K} \sum_{k=1}^K A_k \nabla_{\theta} \log \pi_{\theta}(a_k | x) + \beta \nabla_{\theta} \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\theta_{\text{ref}}}(\cdot | x)).$$

$$\text{reward} = \underbrace{1.0 \cdot \mathbf{1}[\hat{y} = y]}_{\text{accuracy reward}} + \underbrace{0.3 \cdot (\log p(y) - \log p(1 - y))}_{\text{margin confidence reward}} + \underbrace{0.4 \cdot (F1 - 0.5)}_{\text{batch-wide F1 bonus}}$$

$$\text{reward} = \underbrace{1.0 \cdot [\hat{y} = y]}_{\text{accuracy reward}} + \underbrace{0.1 \cdot (\sigma(\log p(y)) - 0.5)}_{\text{soft confidence bonus}}$$

Connection to supervised learning. If we restrict the action space to a single deterministic label token and replace A_k with $(\mathbf{1}[\hat{y} = y^*] - \bar{r})$, GRPO reduces to a variant of weighted cross-entropy with an adaptive, per-group baseline. In practice, however, we allow the model to generate full natural-language justifications, so π_{θ} is optimized over the space of *reasoned judgments* rather than bare labels. The verifiable reward still depends only on the extracted decision $f(a)$, but the KL-regularization term and the language modeling structure of π_{θ} encourage coherent, concise rationales.

Why RLVR helps. Supervised baselines learn to mimic statistical patterns in the training set: they optimize $\log p_{\theta}(y^* | x)$ under a fixed data distribution. In contrast, the RLVR-aligned judge learns a policy that is explicitly rewarded for being *correct* on adversarially difficult and distribution-shifted examples, including paraphrased and humanized AI tweets. GRPO encourages the model to sharpen the decision boundary between human and AI tweets while retaining the general linguistic competence of the base Qwen2.5-14B model. This helps mitigate the over-recall failure mode observed in zero-shot LLM judges and yields a detector that is both more calibrated and more robust in high-stakes political settings.

3.3.7 Practical Implementation Variant

Departing from the theoretical RLVR formulation—where the reward is the binary verifier output $r = \mathbf{1}[f(a) = y^*]$ —our implementation uses a *continuous, shaped reward* designed to stabilize GRPO updates on Qwen2.5 models. For each tweet–label pair (x_i, y_i^*) , let $\hat{y}_i$ denote the model action and let $\log \pi_{\theta}(k | x_i)$ be the classifier log-probability for class k . The practical reward is:

$$r_i = 1.0 \cdot \mathbf{1}[\hat{y}_i = y_i^*] + 0.3 \cdot (\log \pi_{\theta}(y_i^* | x_i) - \log \pi_{\theta}(1 - y_i^* | x_i)) + 0.4 \cdot (F1_{\text{batch}} - 0.5).$$

This design introduces three key deviations from the theoretical RLVR reward:

- (1) a **logit-margin term** encouraging confident, margin-separated predictions;
- (2) a **batch-level F1 shaping term** that biases the policy toward balanced precision–recall behavior;
- (3) a **group-relative advantage** $A_i = r_i - \bar{r}$ replacing the original per-input RLVR baseline.

These modifications preserve the verifiability structure of RLVR while greatly improving gradient stability in practice.

3.4 Model Serving and Infrastructure

TweetVerify is organized as an end-to-end pipeline that connects external data sources, a curated data lake, and a family of detection models. Political tweets from the Twitter API, synthetic political tweets from large language models, and public corpora such as Sentiment140 and DAIGT are normalized and merged into a parquet-backed tweet corpus designed specifically for short-text authorship verification. This corpus drives training for multiple architectures, including RNN/LSTM baselines with Word2Vec embeddings, transformer-based classifiers (BERT, RoBERTa, DeBERTa), and hybrid models with handcrafted stylometric features, with larger training runs offloaded to AWS SageMaker and their resulting checkpoints stored in S3 and synchronized to a central model repository.

On the serving side, a public prediction application and an administrative dashboard run on an AWS EC2 instance, sharing access to the model repository and a PostgreSQL database that stores user accounts and model

metadata. Infrastructure is provisioned via Terraform, which defines the EC2 host, an RDS PostgreSQL instance, and S3 buckets, along with security groups that restrict database access to the application tier. Application-level defenses include per-IP rate limiting for prediction and admin endpoints, strict validation and sanitization of JSON requests, and guarded model upload and selection workflows, providing a hardened but flexible deployment for high-stakes political tweet verification.

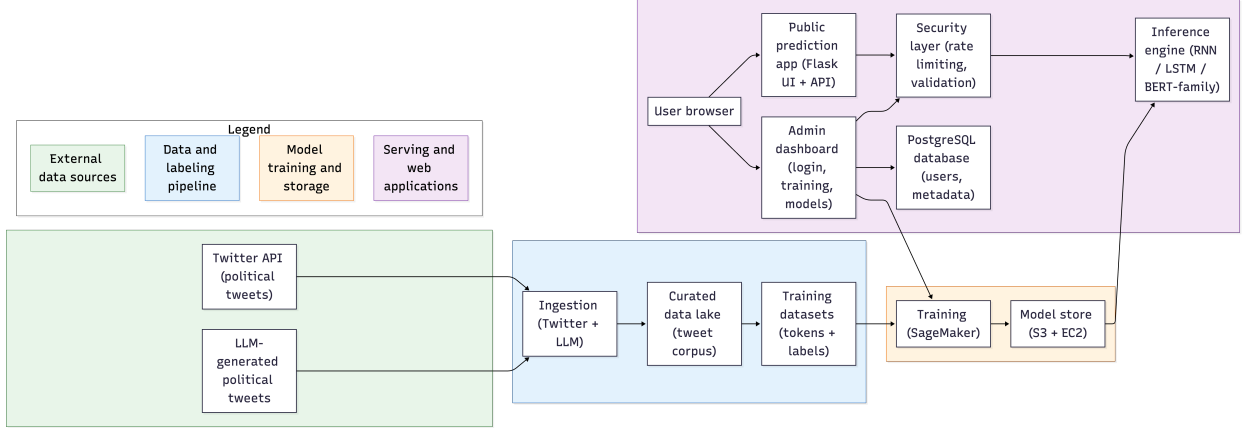


Figure 1: System Architecture and Infrastructure

4 Experiments, key metrics and results

4.1 Training

For the RNN and LSTM baselines, we use the Adam optimizer with a StepLR scheduler ($\text{step_size} = 3$, $\text{gamma} = 0.5$), a batch size of 314, and a learning rate of 1×10^{-3} . For Transformer-based models, we use AdamW with a weight decay of 0.01 and a learning rate of 1×10^{-5} , together with a linear warmup schedule (10% warmup). Gradient clipping with a maximum norm of 3 is applied to stabilize training. All models are trained in a GPU-enabled environment.

For the LLM training, we use Qwen2.5-14B-Instruct as our base model. The model is initialized in mixed precision (bfloat16) and distributed across $8 \times$ A100-40GB GPUs. We adopt the AdamW optimizer for both training stages, with a weight decay of 0.01 to improve generalization. To stabilize early training dynamics, we apply a linear warmup learning-rate scheduler, which gradually ramps up the effective learning rate before transitioning into standard linear decay.

4.2 Benchmarking

To thoroughly test the performance across our various models, we implemented a benchmarking framework that can evaluate the models under consistent conditions.

The dataset used for evaluation comes from a held-out test set that none of the models were exposed to during training. It contains $n = 4,791$ samples, consisting of 3,219 AI-generated texts and 1,572 human-generated texts. This distribution closely mirrors that of the training set and reflects real-world conditions, where AI-generated content is more prevalent than human-written content.

The key metrics we use are the following:

- **Accuracy:** Accuracy measures the overall proportion of correctly classified samples. It is the most intuitive and easy-to-interpret metric for assessing general performance.
- **F1 Score:** The F1 score is the harmonic mean of precision and recall, offering a robust summary of performance under class imbalance where accuracy alone can be misleading.
- **AUC (Area Under the ROC Curve):** AUC is computed from probability outputs and captures the model’s ability to consistently rank AI-generated samples above human-written ones, independent of any particular decision threshold.

We report the average performance over five runs. For the ensemble model, we combine BERT, RoBERTa, and DeBERTa with weights of 0.3, 0.1, and 0.9, respectively.

Model	Accuracy	F1 Score	AUC
RNN	0.674 ± 0.003	0.805 ± 0.002	0.581 ± 0.007
LSTM	0.682 ± 0.003	0.808 ± 0.002	0.638 ± 0.008
Zero-Shot	0.621 ± 0.009	0.732 ± 0.006	0.550 ± 0.010
BERT	0.756 ± 0.002	0.840 ± 0.002	0.718 ± 0.005
DeBERTa	0.757 ± 0.003	0.834 ± 0.003	0.753 ± 0.003
RoBERTa	0.835 ± 0.003	0.879 ± 0.003	0.893 ± 0.002
RoBERTa (Feature-Enhanced)	0.831 ± 0.002	0.876 ± 0.001	0.881 ± 0.002
Qwen2.5-7B SFT&GRPO	0.751 ± 0.003	0.786 ± 0.003	0.837 ± 0.003
Ensemble	0.844 ± 0.002	0.888 ± 0.002	0.870 ± 0.003

Table 1: Performance of different models (mean $\pm$ standard deviation over five runs).

4.3 Results and Analysis

As shown in the table, the transformer-based models clearly outperform all baseline methods, with the ensemble model achieving the best overall performance. This result is expected, as weighted averaging across several strong candidate models allows the ensemble to combine their strengths while reducing individual model biases.

4.3.1 Zero-Shot Method

In our experiment, the zero-shot method did not achieve strong performance. This is partly because the sampling model we used—GPT-2 XL—cannot accurately represent the token distribution of modern LLMs such as GPT-5, which generated many of the tweets. Additionally, the short length of tweets provides limited information for curvature estimation, leading to biased results. We acknowledge this as a limitation and will discuss it further in the Next Steps section.

4.3.2 Feature-Augmented RoBERTa Model

From the results, the *Feature-Enhanced RoBERTa* model did not significantly outperform the plain RoBERTa baseline. To probe this, we compared the distributions of hand-crafted features between human- and AI-generated tweets (Fig. 2).

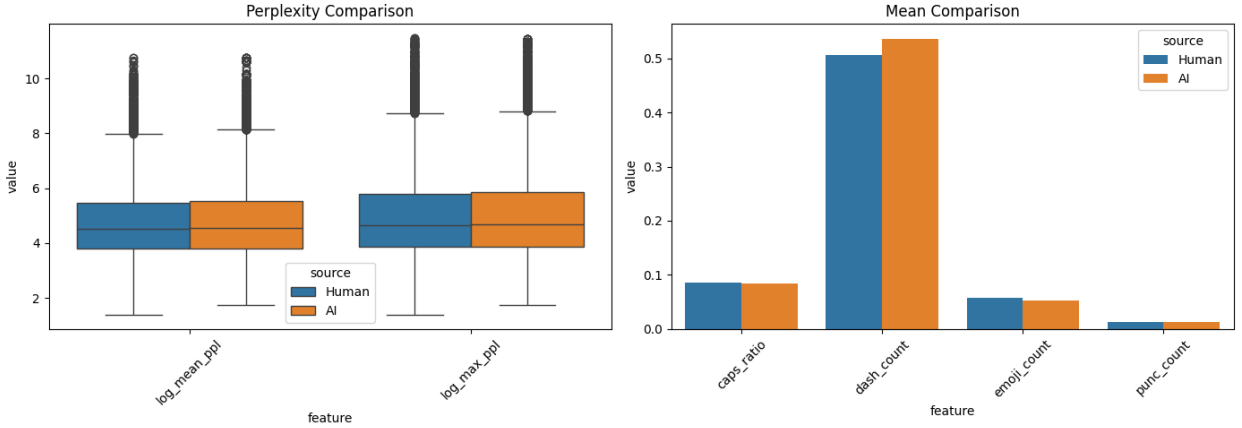


Figure 2: Distributions of selected features for human- vs. AI-generated tweets.

As shown in Fig. 2, the differences are modest: AI-generated tweets exhibit slightly higher perplexity and dash count, whereas other features provide little discriminatory signal. This may be due to two factors. First, state-of-the-art generators produce text that closely mimics human style, reducing separability in surface-level features. Second, tweets are short, providing limited context for these features to diverge meaningfully.

4.3.3 Qwen2.5 Models (7B/14B Judges)

From Table 1, we observe that our Qwen2.5-7B-Instruct judge—both in its supervised fine-tuning (SFT) form and in the GRPO-aligned RLVR variant—does not outperform the best encoder-only baselines such as RoBERTa and the ensemble. We also conducted limited experiments with a larger Qwen2.5-14B-Instruct judge using the same

SFT+GRPO pipeline; its behaviour was qualitatively similar, confirming that simply scaling up the autoregressive backbone does not close the gap to masked-language-model encoders on this short-text detection task.

Our dataset consists of political tweets generated by advanced LLMs such as GPT-4o-mini and GPT-5-mini. Although these models produce human-like text at the stylistic level, prior work has repeatedly shown that machine-generated content still carries subtle but systematic *statistical artifacts*. These include smoother token distributions, reduced syntactic variance, and unusually regularized function-word patterns, all of which distinguish AI-generated text from human writing even when stylistic fluency is high. This observation is consistent with findings in the detection literature: DetectGPT Mitchell et al. (2023) demonstrates that curvature-based deviations in log-probability space provide a reliable low-level signal, while DetectLLM Su et al. (2023) shows that token rank instability is a strong indicator of LLM-generated text independent of surface style.

MLM-based encoders (e.g., BERT) are inherently sensitive to such token-level irregularities due to their pre-training objective, which models local distributional structure rather than global semantic coherence. As a result, these models detect the fine-grained statistical anomalies present in modern LLM-generated tweets and substantially outperform autoregressive LLMs on this task. Our results echo the findings of Miresghallah et al. (2024), who show that smaller language models—especially masked or encoder-only architectures—serve as more effective zero-shot detectors than large generative models.

In contrast, large autoregressive models such as Qwen2.5-7B operate in a fundamentally different regime: their internal representations are dominated by *semantic* and *stylistic* factors. When applied to AI-human classification, they tend to rely on high-level discourse cues (e.g., coherence, politeness, argumentative balance) rather than distributional deviations. Because GPT-4o-mini and GPT-5-mini already mimic human political style extremely well, these semantic cues are effectively uninformative, resulting in the poor performance in LLMs. Similar limitations have been documented in broader evaluations of AI-content detectors (e.g., Elkhatat et al. (2023)), which note that semantic fluency alone is insufficient for robust detection.

Interpretation. Taken together, these results reinforce an emerging conclusion in AI-generated text detection:

strong generative models $\nrightarrow$ automatically strong detectors

Strong generative models’ semantic richness tends to wash out the shallow token-level irregularities that earlier systems produced—for example, differences in word/phrase frequencies, sentence-length patterns, punctuation behavior, and recurrent “safe” stylistic templates introduced by RLHF or decoding heuristics. These residual distributional artifacts, though semantically irrelevant, remain statistically stable in political tweet corpora and can be efficiently captured by smaller MLM-based classifiers. Because MLMs focus almost exclusively on such surface-level statistical signatures, rather than deeper semantic coherence, they often outperform large LLM judges in machine-generated-vs-human tweet discrimination. In short, the very semantic expressiveness that makes LLMs more human-like in generation becomes a liability in discrimination tasks that hinge on subtle but persistent token-distribution cues.

Implications for System Design. These observations suggest that practical detection systems should either (i) rely on token-level models such as RoBERTa, which directly model distributional irregularities, or (ii) augment large LLMs with explicit statistical features (e.g., per-token log-probabilities, entropy traces, or curvature-based scores Mitchell et al. (2023) and Su et al. (2023)), since style-only reasoning is insufficient against modern generation models.

4.3.4 Overfitting Prevention in Transformer Baselines

All Transformer-based supervised baselines (RoBERTa, RoBERTa-Extra, and DeBERTa-v3) incorporate several architectural and training-level mechanisms to reduce overfitting on the political tweet dataset.

First, each model applies **multiple dropout layers** (rate = 0.1) on top of the [CLS] representation and within the classifier MLP, reducing the chance that the classifier memorizes token-specific patterns. Second, the classifier head is intentionally kept **shallow** (two-layer MLP with ReLU), which limits model capacity and prevents it from overfitting the small input domain of short tweets. Third, all models are trained using **AdamW with weight decay**, which penalizes large weights and improves generalization.

For RoBERTa-Extra, the handcrafted features pass through an additional Dropout-MLP block before being concatenated with the RoBERTa embedding, ensuring that the extra features cannot induce overfitting through high-variance signals. Finally, all baselines rely on **pretrained Transformer encoders**, which provide strong linguistic priors and greatly reduce the risk of memorizing dataset-specific artifacts. Together, these design choices ensure stable validation performance and prevent the small models from overspecializing on the training distribution.

4.4 Comparison With Existing Models

We also compared our model against state-of-the-art systems to assess whether it can outperform them. Specifically, we evaluated GPT-5.1 Thinking, Gemini 3 Pro, and ZeroGPT. Based on the results from the previous section, we used our ensemble model, as it achieved the highest overall performance across all metrics.

For this evaluation, we randomly selected 50 samples from the test set, consisting of 14 human-generated tweets and 36 AI-generated tweets.

Model	Accuracy	Precision	Recall
Our Best Model	0.86	0.87	0.94
GPT-5.1 Thinking	0.56	0.77	0.56
Gemini 3 Pro	0.58	0.74	0.64
ZeroGPT	0.28	0	0

Table 2: Performance comparison on the tweet classification task.

From the table, we observe that our best ensemble model outperforms all of the SOTA baselines, demonstrating its effectiveness. It is also worth noting that ZeroGPT failed to identify any AI-generated tweets, which may be due to the short length of the tweets or limited exposure of the model to this specific domain.

5 Future work and next steps

5.1 Modelling

In terms of modeling, we aim to improve the performance of our system further to provide more reliable and accurate results for end users.

For the zero-shot method, our current implementation does not achieve satisfactory performance. As a next step, we plan to adopt stronger sampling models (e.g., Qwen2.5-7B/14B and GPT-4o-mini) that better match the token distributions of modern generators such as GPT-5. In addition, we will explore adaptive perturbation strategies that adjust perturbation size based on input length, as well as simple post-hoc scaling of curvature scores to make them more stable on very short texts. Together, these improvements aim to increase robustness and enhance zero-shot detection accuracy.

Tweets also often include hashtags that indicate events or topics, “@” mentions that reference specific users, and embedded links that point to external content. These components contain additional contextual information that could help the classifier better understand the intent, audience, or thematic focus of a tweet. Despite this potential, our current implementation does not make meaningful use of these elements: they are either treated as ordinary tokens or removed during preprocessing. In future work, we plan to investigate techniques for extracting richer features from these components, for example, normalizing and grouping hashtags by topic clusters, identifying user roles or communities from mentions, and incorporating link metadata as auxiliary signals.

5.2 Data and Evaluation

On the data side, our current setup focuses on English political tweets from a single public corpus plus LLM-generated variants; extending this to additional languages, regions, and election cycles would allow us to test robustness under genuine distribution shift. Moreover, we plan to build a small, manually checked test set that contains especially difficult cases, such as sarcasm, meme-style tweets, and tweets that mix copied and lightly edited AI output. This kind of “hard” benchmark would help us understand concrete failure modes rather than only looking at average accuracy. We also aim to periodically add tweets from new political events and newly released LLMs, so that our evaluation stays up to date with the kinds of content and generation patterns the system will see in practice.

6 Team reflection

The development of TweetVerify over the course of this term has been a transition from a theoretical research interest into a robust, production-grade engineering system. Through the structure of CSC490, our team evolved our perspective on Machine Learning Engineering, moving beyond model accuracy as the sole metric of success to embrace the broader challenges of infrastructure reliability, data stewardship, and system scalability.

6.1 Connection to Course Material

Cloud Infrastructure and the 12-Factor App (Lecture 2): Assignment A3 challenged us to codify our infrastructure using Terraform. This directly applied the lecture principles regarding *Infrastructure as Code (IaC)* and the elimination of "it works on my machine" issues. By treating our AWS infrastructure (EC2, RDS, S3) as immutable code, we eliminated configuration drift between our development and production environments. Furthermore, our containerization strategy adhered to the *12-Factor App* methodology discussed in class, strictly separating config from code and treating backing services (like our PostgreSQL database) as attached resources. This practice was vital during our disaster recovery simulations, allowing us to tear down and rebuild our staging environment in minutes.

Model Serving and the Latency-Throughput Trade-off (Lecture 5): The load testing module (Assignment A5) forced us to confront the theoretical constraints of model serving discussed in Lecture 5. While our initial Transformer models achieved high accuracy, the *Time to First Token (TTFT)* and total inference time were unacceptable for a real-time social media tool. Applying the lecture concepts of *horizontal scaling* versus *vertical scaling*, we realized that throwing larger hardware at the problem (vertical) was less cost-effective than optimizing the serving layer. We implemented specific optimizations discussed in class, such as vectorization to remove CPU bottlenecks in preprocessing, and introduced rate-limiting middleware to manage throughput saturation, shifting our focus from pure accuracy to strict SLA compliance.

Data-Centric AI and Transfer Learning (Lectures 7 & 9): The course emphasis on *Data-Centric Engineering* resonated deeply during our pipeline construction (Assignment A2). As discussed in Lecture 7, modern ML engineering is often less about architectural innovation and more about data quality. We moved away from training from scratch and instead leveraged *Transfer Learning* by fine-tuning pre-trained BERT/RoBERTa models. We found that the quality of our political detection system was bounded strictly by the integrity of our data lake. Implementing robust schemas and cleaning pipelines (as emphasized in the Feature Store/Data Engineering lectures) allowed us to train models that generalized to unseen political contexts rather than memorizing noise in the training set.

6.2 Technical Evolution and Lessons Learned

Our most significant technical realization occurred during the integration of RLVR. In Assignment A4, drawing on Lecture 8's exploration of RLHF and PPO, we hypothesized that aligning an LLM-as-a-judge via GRPO would yield the ultimate detector. However, our empirical results painted a different picture: while RLVR improved the *reasoning* capabilities of Qwen2.5, it could not outperform smaller, specialized encoder models (BERT/RoBERTa) at detecting the subtle statistical artifacts of AI generation in short-text.

This taught us a valuable lesson in **right-sizing the solution**. In the hype cycle of Generative AI, there is a temptation to apply Large Language Models to every problem. Our project demonstrated that for specific discriminative tasks—particularly those relying on token-level distribution irregularities rather than semantic logic—specialized, lower-latency architectures are often superior. Consequently, our final solution pivoted to an ensemble approach, prioritizing the efficiency and precision of encoders while reserving the LLM judge for experimental, high-level reasoning tasks.

6.3 Team Dynamics and Future Outlook

Our team effectively balanced the dual demands of research and engineering. While one sub-team focused on the aggressive exploration of adversarial attacks and RLVR tuning, the other focused on hardening the API and ensuring test coverage (Assignment A6). This parallel workflow allowed us to innovate without destabilizing the core product.

Looking forward, we recognize that AI detection is an adversarial arms race. As generation models like GPT-5 become more sophisticated, the statistical signals our current models rely on will diminish. This project has highlighted that long-term viability requires not just a static model, but a continuous training pipeline, which is one that we have now successfully built, tested, and deployed.

References

- Bao, G., Zhao, Y., Teng, Z., Yang, L., & Zhang, Y. (2024). Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature. <https://arxiv.org/abs/2310.05130>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. <https://arxiv.org/abs/1810.04805>
- Elkhatat, A. M., Elsaid, K., & Almeer, S. (2023). Evaluating the efficacy of AI content detection tools in differentiating between human and AI-generated text. *International Journal for Educational Integrity*, 19(1), 17. <https://doi.org/10.1007/s40979-023-00140-5>
- Kumarage, T., Garland, J., Bhattacharjee, A., Trapeznikov, K., Ruston, S., & Liu, H. (2023). Stylometric detection of ai-generated text in twitter timelines. <https://arxiv.org/abs/2303.03697>
- Mireshghallah, N., Mattern, J., Gao, S., Shokri, R., & Berg-Kirkpatrick, T. (2024). Smaller language models are better zero-shot machine-generated text detectors. <https://doi.org/10.18653/v1/2024.eacl-short.25>
- Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., & Finn, C. (2023). Detectgpt: Zero-shot machine-generated text detection using probability curvature. <https://arxiv.org/abs/2301.11305>
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. <https://arxiv.org/abs/2402.03300>
- Su, J., Zhuo, T. Y., Wang, D., & Nakov, P. (2023). Detectllm: Leveraging log rank information for zero-shot detection of machine-generated text. <https://arxiv.org/abs/2306.05540>
- Suresh, K. (2021). Global political tweets [[Data set] <https://www.kaggle.com/datasets/kaushiksuresh147/political-tweets>]. <https://www.kaggle.com/datasets/kaushiksuresh147/political-tweets>
- Wen, X., Liu, Z., Zheng, S., Ye, S., Wu, Z., Wang, Y., Xu, Z., Liang, X., Li, J., Miao, Z., Bian, J., & Yang, M. (2025). Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. <https://arxiv.org/abs/2506.14245>
- Zhao, Y., Liu, H., Yu, D., Kung, S., Chen, M., Mi, H., & Yu, D. (2025). One token to fool llm-as-a-judge. <https://arxiv.org/abs/2507.08794>